ULB CONTINUING EDUCATION HELSCI HEALTH & LIFE SCIENCES

Data Collection Project

M. Andrin, O. Bevz, P. Brunet, C. Simeu

Academic year 2025–2026

Content

1 Introduction

The goal of this project is to build a small but complete web application that connects to the Fitbit API to retrieve real user activity data, such as steps, distance and other health-related metrics.

Through this project, we want to understand how real-word APIs work, especially those that require secure authentication through OAuth 2.0, which is the standard method used by most modern platforms like Google, Fitbit, Facebook, to protect user data.

We used Flask, a lightweight Python web framework, to create the web interface and handle the authentication process. Once the user logs in with their Fitbit account, the app retrieves an access token and uses it to request their daily activity data through Fitbit's REST API.

The project also used *ngrok* to expose the Flask app to the internet, allowing Fitbit's OAuth system to communicate with the local development server. The retrieved data is displayed in a clear, visual dashboard built with *Chart.js*, allowing users to select start date and end date, and immediately view their statistics.

This work demonstrates our ability to:

- Integrate an external API into a Python application
- Manage OAuth 2.0 authentication flows,
- Handle API data and display it visually,
- Troubleshoot common integration problems such as invalid tokens, redirect URL, mismatches, or expired codes.

In short, the project simulates what happens when a real-world application – like a fitness tracker or analytics dashboard – connects securely to a user's online account to personalize their experience.

During the development process, it became clear that the Fitbit developer account used for testing did not have any synchronized activity data, which made it impossible to display meaningful analytics directly from the API. Instead of leaving the dashboard empty, we decided to use a real-world dataset from Kaggle that mirrors Fitbit's data structure and metrics. This choice allowed us to:

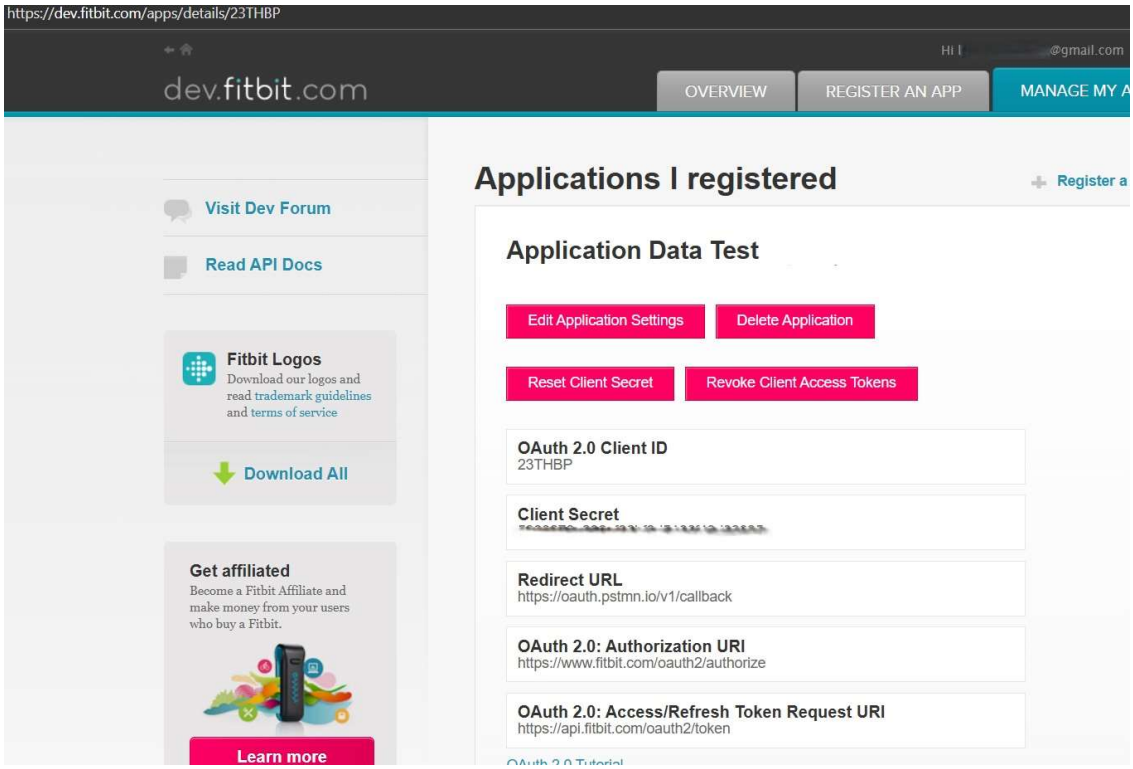
- Keep the same data format and logic as the Fitbit API responses,
- Develop and test all parts of the ETL and visualization pipeline,
- Demonstrate a fully functional dashboard capable of analyzing fitness data
- Simulate how the application would behave once connected to a real user account.

By making this adaptation, we ensured the project not only achieved its technical goals, but also delivered a functional data-driven dashboard demonstrating all core functionalities of the intended system.

2 Fitbit Developer App Registration

2.1 Create a Fitbit developer account

We created a Fitbit developer account (at https://dev.fitbit.com) to obtain API credentials (Client **ID** and Client **Secret**).



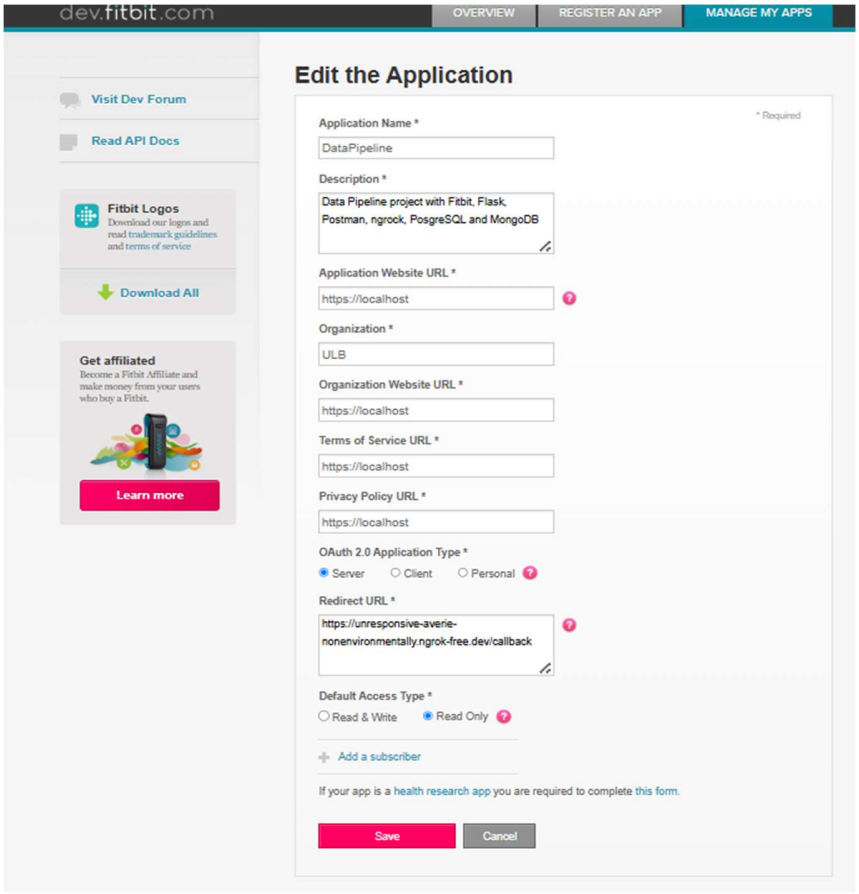
This credentials uniquely identify our application to Fitbit's API and are required to authenticate during the OAuth 2.0 flow. There are used to request access tokens and refresh tokens securely.

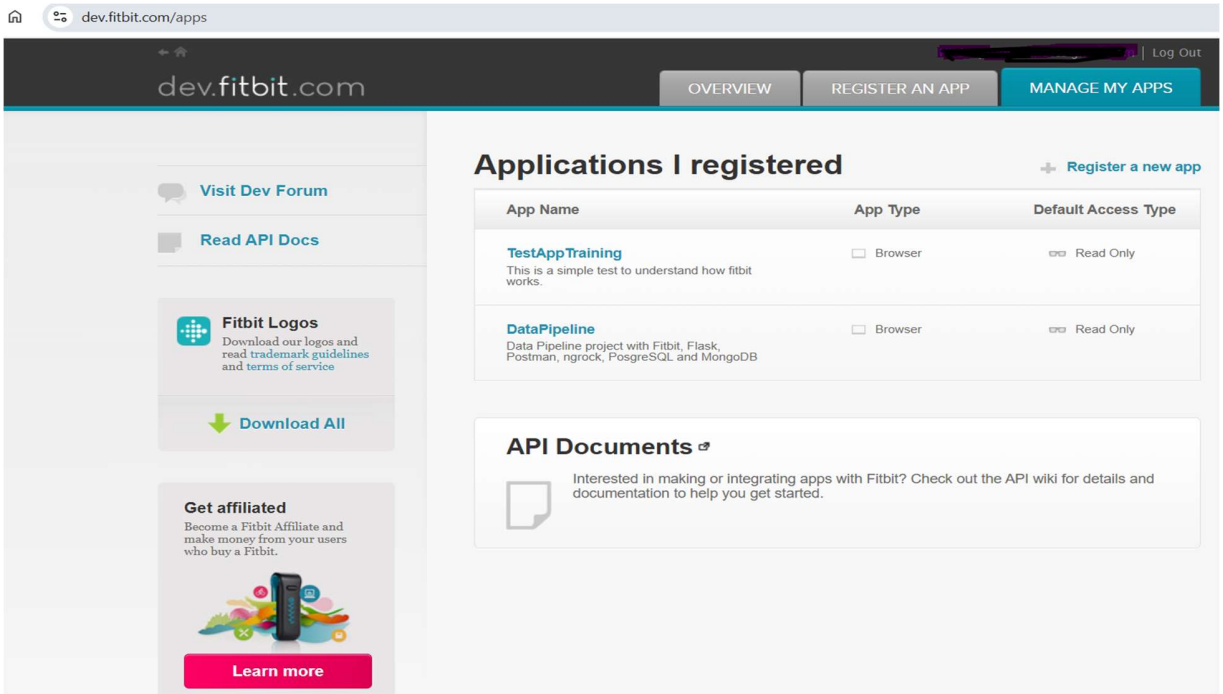
2.2 App registration on Fitbit Developer Portal

We have registered a new application on Fitbit Developer Portal with the following configuration:

- Application Type: **Server**
- Redirect URL: [First *Postman callback*, then *ngrok callback*]
- Scopes: activity heartrate, sleep profile

This step links our app (Flask app, Postman app etc…) to Fitbit's system and defines the scopes it can request from users. It allows Fitbit to recognize our app and authorize it to access user data securely. The redirect URL is used by Fitbit to send the authorization code back to our app after a successful login.


dev.fitbit.com
OVERVIEW
REGISTER AN APP
MANAGE MY APPS
Visit Dev Forum
Read API Docs
Fitbit Logos
Download our logos and read trademark guidelines and terms of service
Download All
Get affiliated
Become a Fitbit Affiliate and make money from your users who buy a Fitbit.
Learn more
Edit the Application
* Required
Application Name *
DataPipeline
Description *
Data Pipeline project with Fitbit, Flask, Postman, ngrock, PosgreSQL and MongoDB
Application Website URL *
https://localhost
Organization *
ULB
Organization Website URL *
https://localhost
Terms of Service URL *
https://localhost
Privacy Policy URL *
https://localhost
OAuth 2.0 Application Type *
Server
Client
Personal
Redirect URL *
https://unresponsive-averie-nonenvironmentally.ngrok-free.dev/callback
Default Access Type *
Read & Write
Read Only
Add a subscriber
If your app is a health research app you are required to complete this form.
Save
Cancel

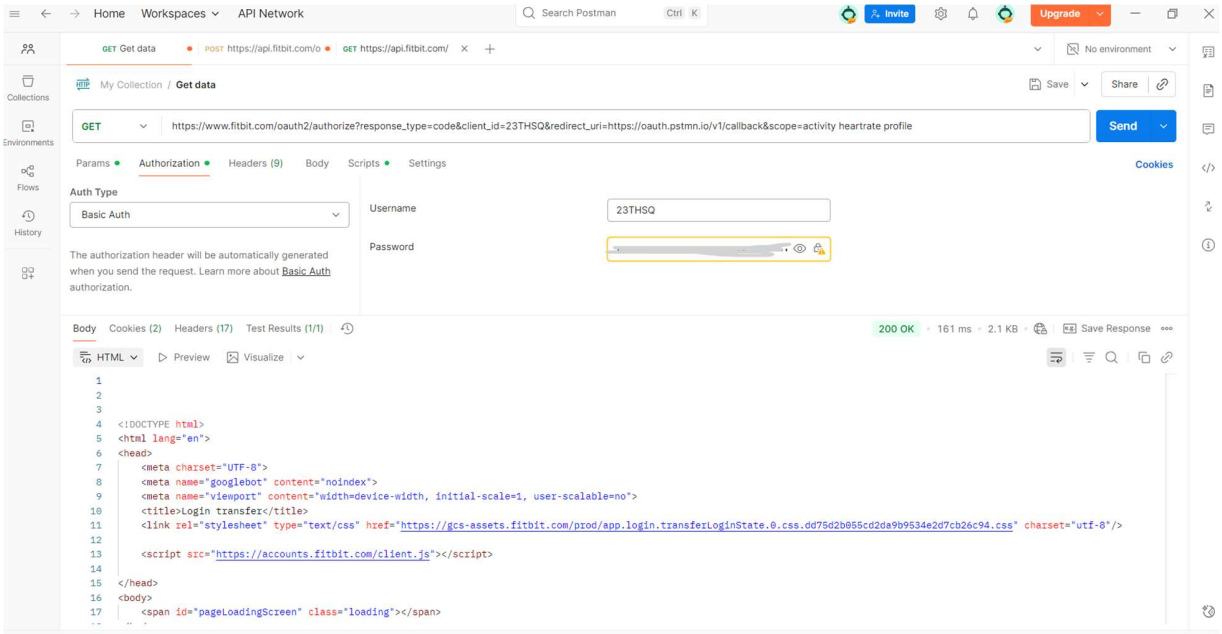

dev.fitbit.com/apps
Log Out
dev.fitbit.com
OVERVIEW
REGISTER AN APP
MANAGE MY APPS
Visit Dev Forum
Read API Docs
Fitbit Logos
Download our logos and read trademark guidelines and terms of service
Download All
Get affiliated
Become a Fitbit Affiliate and make money from your users who buy a Fitbit.
Learn more
Applications I registered
Register a new app
App Name
App Type
Default Access Type
TestAppTraining
This is a simple test to understand how fitbit works.
Browser
Read Only
DataPipeline
Data Pipeline project with Fitbit, Flask, Postman, ngrock, PosgreSQL and MongoDB
Browser
Read Only
API Documents
Interested in making or integrating apps with Fitbit? Check out the API wiki for details and documentation to help you get started.

2.3 Exchanging Authorization code for Access and Refresh Tokens in Postman

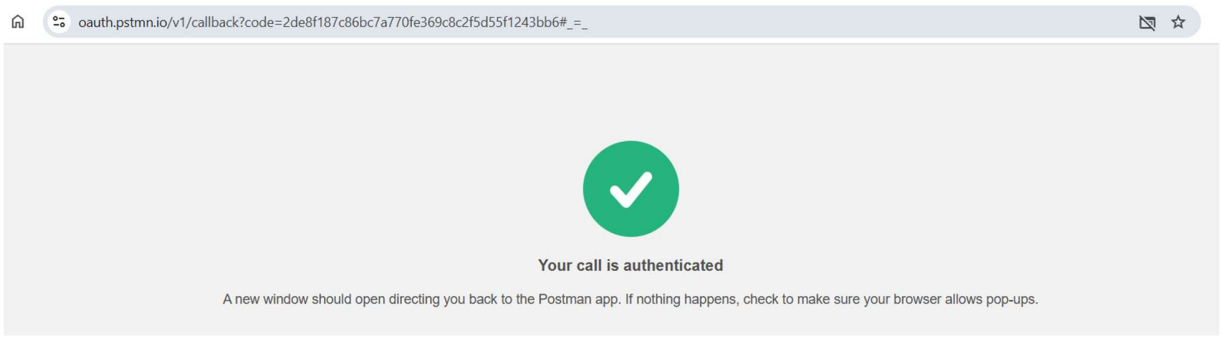
We used Postman to manually perform the OAuth token exchange before automating it in Flask. This step converts the authorization code retrieved from Fitbit after user login, into an access token and refresh token. It involves two main requests steps (GET and POST):

1. A **GET request** to get an authorization code.
   In order to ask Fitbit for a permission to access user data, a GET request performed in Postman and before doing that, we replace our redirect URL on Fitbit with https://oauth.pstmn.io/v1/callback

   This screen is the result of our GET request.

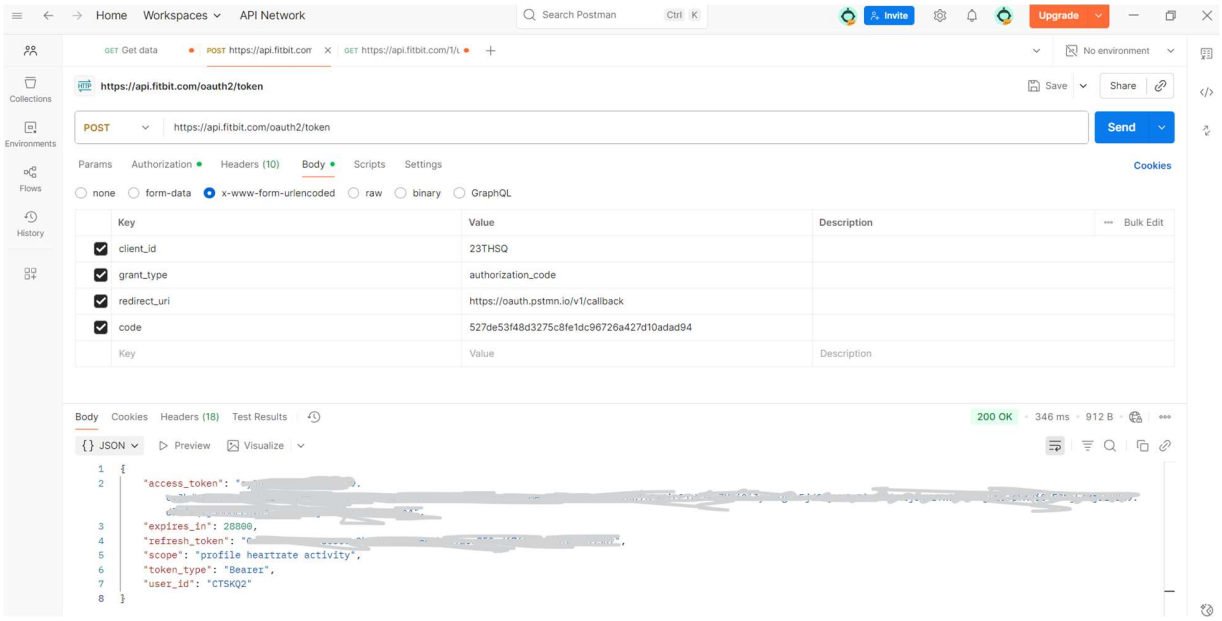


   The following screen shows the code displayed after the GET request. This will be used in the POST request to obtain Access and Refresh Tokens.



2. A **POST request** to exchange the code obtained above with access and refresh tokens.

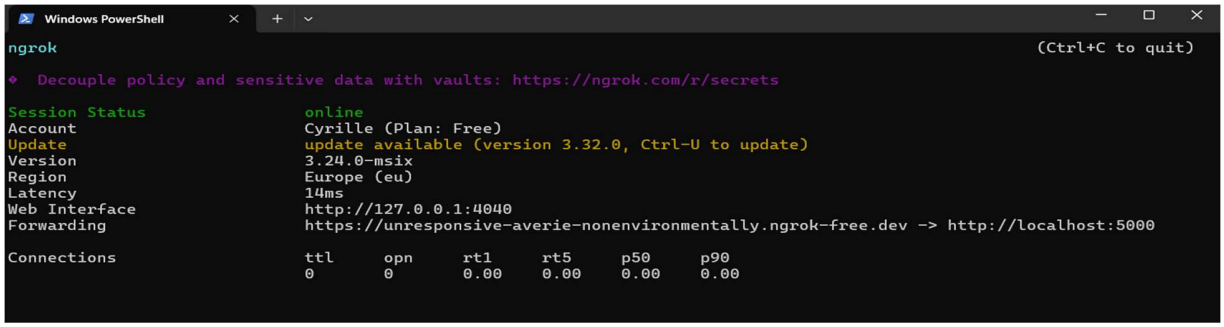
   This screen shows the result of the POST request. The Access and Refresh tokens provided by Fitbit will be inserted into the .env file to handle the connection and exchange automatically.



3 Ngrok Configuration

Ngrok was used to make the local Flask server accessible over the internet. After installing and authenticating ngrok with an authtoken, a secure HTTPS tunnel was created to the local port 5000 (the port used by Flask). The public URL generated by ngrok was then used as the redirect URL for the Fitbit OAuth 2.0 configuration.

This screen shows the ngrok configuration on Windows PowerShell:

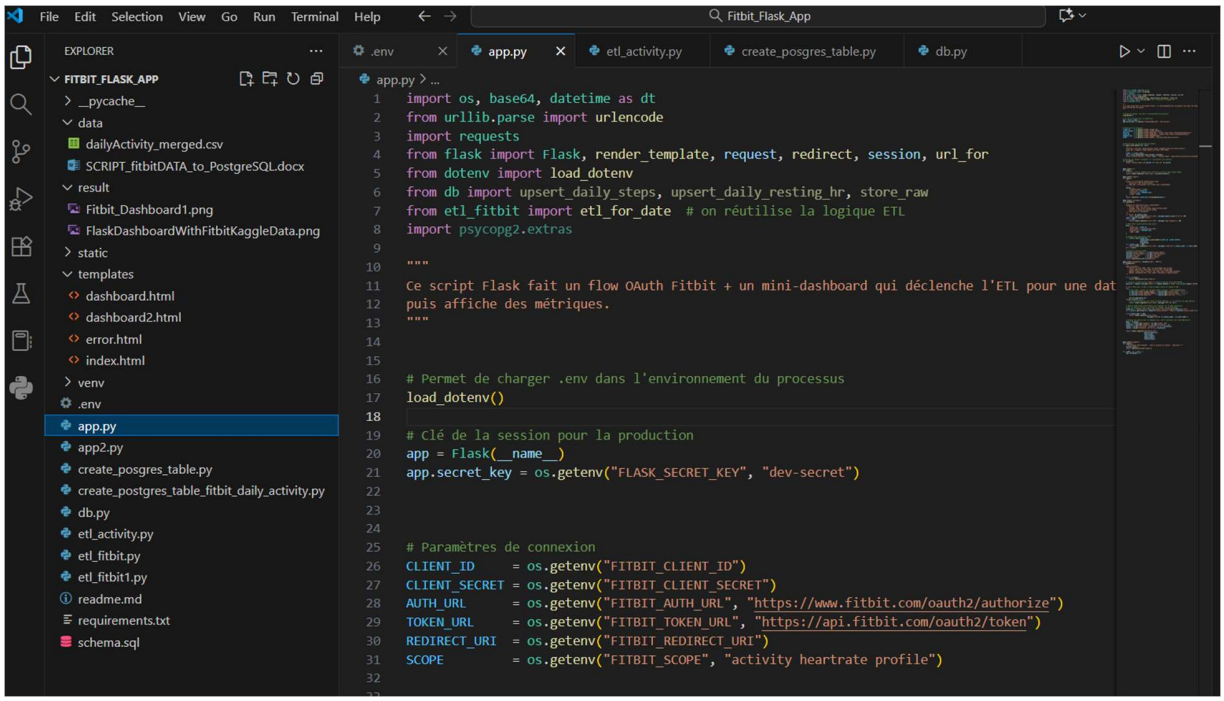


4 Application Architecture

This project follows a modular architecture designed to separate concerns between authentication, data retrieval, storage, and visualization. It combines a Flask web interface, a Fitbit API connection, and a PostgreSQL database to simulate real-world data processing and analytics.

4.1 Project Structure

The overall project structure is as follows:



This structure keeps the code organized and allows each part of the system to evolve independently.

4.2 The .env file

The .env file is used to securely store all the environment variables required by the Flask application and the database. It contains sensitive information – such as API credentials, database connection details, and callback URLs – which should never be hardcoded directly into the code. Typical variables stored in this file include:

- FITBIT_CLIENT_ID: the Fitbit client ID.
- FITBIT_CLIENT_SECRET: the client secret.
- FITBIT_REDIRECT_URI: the URL where Fitbit redirects after login for the OAuth 2.0 process.
- ACCESS_TOKEN: Temporary authorization token obtained after a successful OAuth 2.0 login.

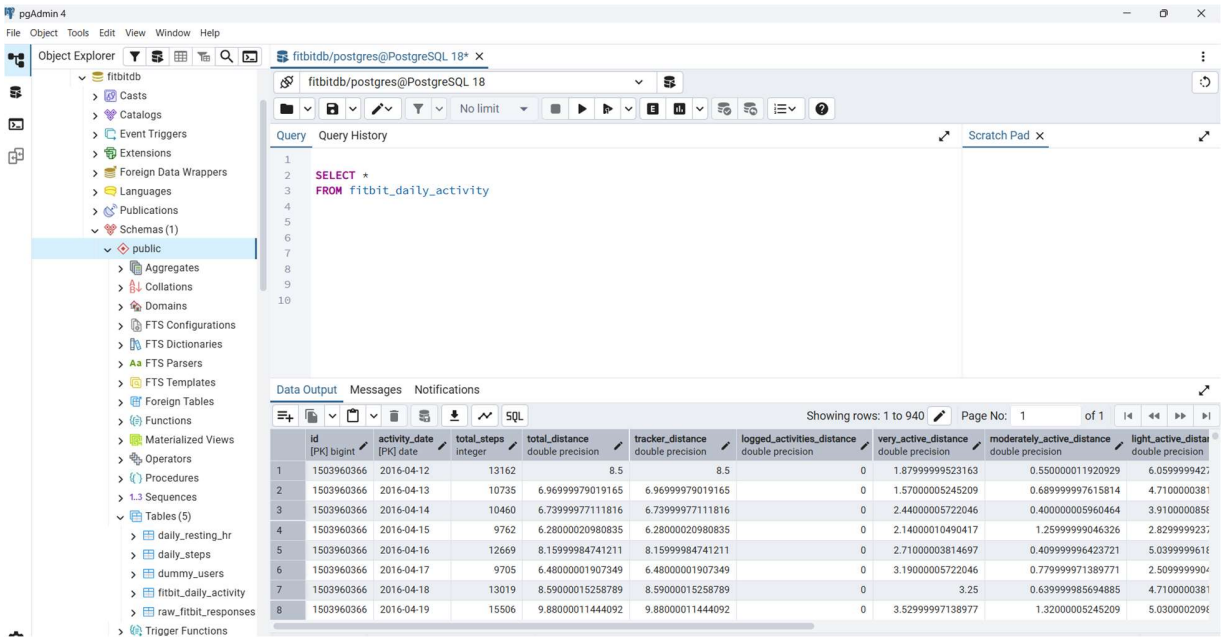
- REFRESH_OKEN: Long-lived token that can be used to get a new access token when the previous one expires.
- USER_ID: Unique identifier assigned by Fitbit to the authenticated user.
- PG_HOST: Host address of the PostgreSQL server (in our case, it is localhost).
- PG_USER: Username used to connect to PgAdmin 4.
- PG_PASSWORD: Password associated with the PgAdmin 4 user.
- PG_DB: Name of the Postgres database, in our case it is “fitbitdb”.
- FLASK_SECRET_KEY: used to manage secure user sessions in the web app.

4.3 The script: create_posgres_table.py

This Python script is responsible for creating the necessary PostgreSQL database tables used by the Fitbit Flask application. It connects to the database “fitbitdb” using the connection parameters stored in the .env file and automatically creates the tables if they do not already exist.

The main tables created are:

- **daily_steps:** to store the number of steps per user and date.
- **Daily_resting_hr:** to store each user’s daily resting heart rate.
- **Raw_fitbit_responses:** to store the full raw JSON data returned by the fitbit API for traceability and debugging.



4.4 The script: create_postgres_table_fitbit_daily_activity.py

This script loads the Fitbit Kaggle dataset from a CSV file, cleans it, and import it into a PostgreSQL database "fitbitdb". It first reads the data, renames the columns from consistency, converts dates to the right format, fills missing values, and removes duplicates. Then it connects to the PostgreSQL database, creates the table "fitbit_daily_activity" with the proper structure for Fitbit metrics if it doesn't already exist, and inserts all the cleaned rows.

4.5 The script: etl_fitbit.py

This script automates the data extraction, transformation, and loading process between the Fitbit API and the PostgreSQL database. It connects securely to Fitbit using OAuth 2.0 tokens, retrieves the user's daily activity and heart rate data, and stores it in the database. This script acts as a bridge between Fitbit and database, ensuring that the latest user activity and the heart rate data are securely fetched, cleaned and stored. It enables Flask dashboards to display real, up-to-date Fitbit insights.

4.6 The script: etl_activity.py

This Python script is used to load user activity data from the PostgreSQL database into a Pandas dataframe for analysis and visualization. It connects the database using the credentials stored in the .env file and executes a SQL query that retrieves daily activity metrics, such as total steps, distance walked, calories burned. The script provides reusable functions to retrieve structured Fitbit activity data from the PostgreSQL database, enabling the Flask application or other scripts to display daily trends and summaries.

4.7 The script: *app.py*

The app.py Flask application connects a user to their Fitbit account, retrieves activity data, and displays it in a simple web dashboard. The process starts when the user clicks on "Login with Fitbit". The app redirects the user to Fitbit's login page using the OAuth 2.0 flow. Once the user grants permission, Fitbit sends back an authorization code, which the app exchanges for access and refresh tokens. These allow the app to securely access the user's activity data. After authentication, the app triggers an ETL process that collects Fitbit data for a chosen date, processes it, and stores it in PostgreSQL database. The dashboard displays useful metrics such as daily steps and distances through an interactive chart.

4.8 The script: *app2.py*

This Flask script builds an interactive dashboard that displays daily activity date from the Kaggle Fitbit dataset "fitbit_daily_activity" stored in PostgreSQL. It connects to the database "fitbitdb", retrieves filtered data based on user input (date range and user ID) and visualizes it using Plotly Charts. It transforms raw activity data into interactive visual insights, allowing users to easily explore their daily fitness patterns directly in the browser.

5 Data Pipeline Overview

5.1 Pipeline components

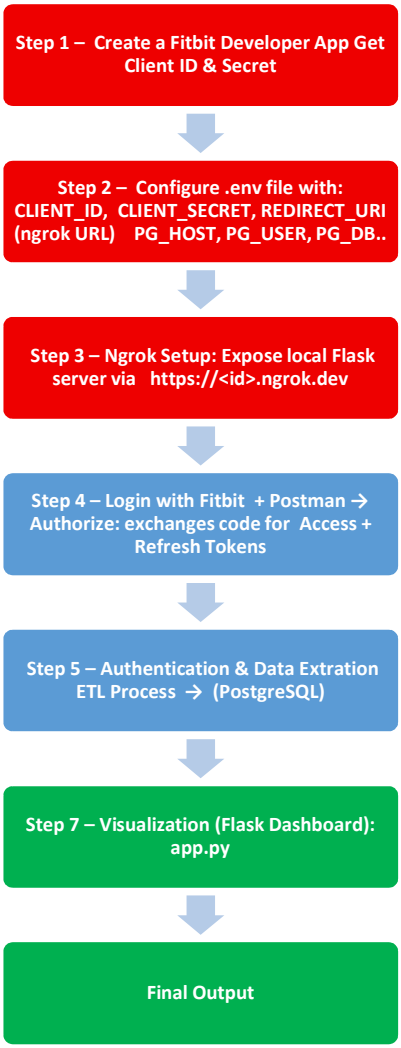
Here are the components of our pipeline:

- Fitbit API (Data source).
- ETL Scripts (Data extraction and loading).
- PostgreSQL Database.
- Ngrok.
- Flask Application (Data visualisation).

5.2 Workflow

The figure below summarizes the complete workflow followed in this project, from configuration to dashboard visualization.

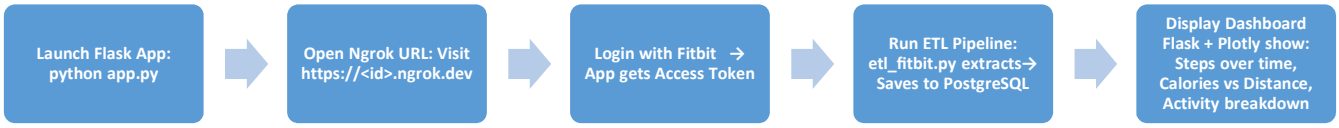
The process begins with the Fitbit Developer setup, where the application credentials are created and stored securely in the .env file. Then Ngrok is used to expose the local Flask application through a public HTTPS URL, enabling Fitbit to communicate with it during the OAuth 2.0 authentication process. Once the user logs in with Fitbit and grants permission, Flask exchanges the authorization code for access and refresh tokens, allowing secure API access. The ETL script then extracts activity and heart rate data from the Fitbit API, transforms it, and loads it into the PostgresSQL database. Finally, the Flask dashboard retrieves this stored data, processes it with Pandas, and visualizes it through interactive Plotly charts.



In general, the **entire process** can be described as follows:

SETUP ▸ AUTHENTICATION & ETL ▸ VISUALIZATION

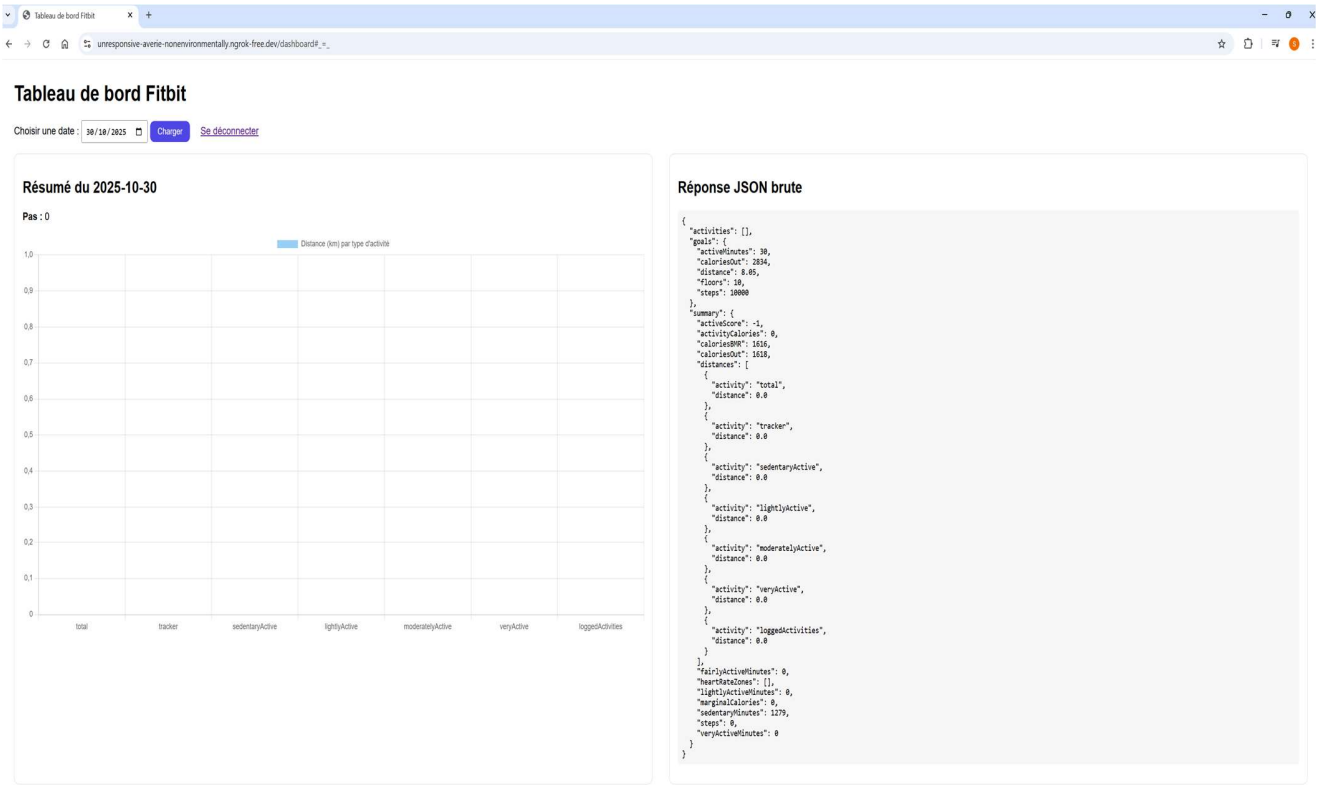
5.3 Run app.py



6 Visualization: Flask Dashboards

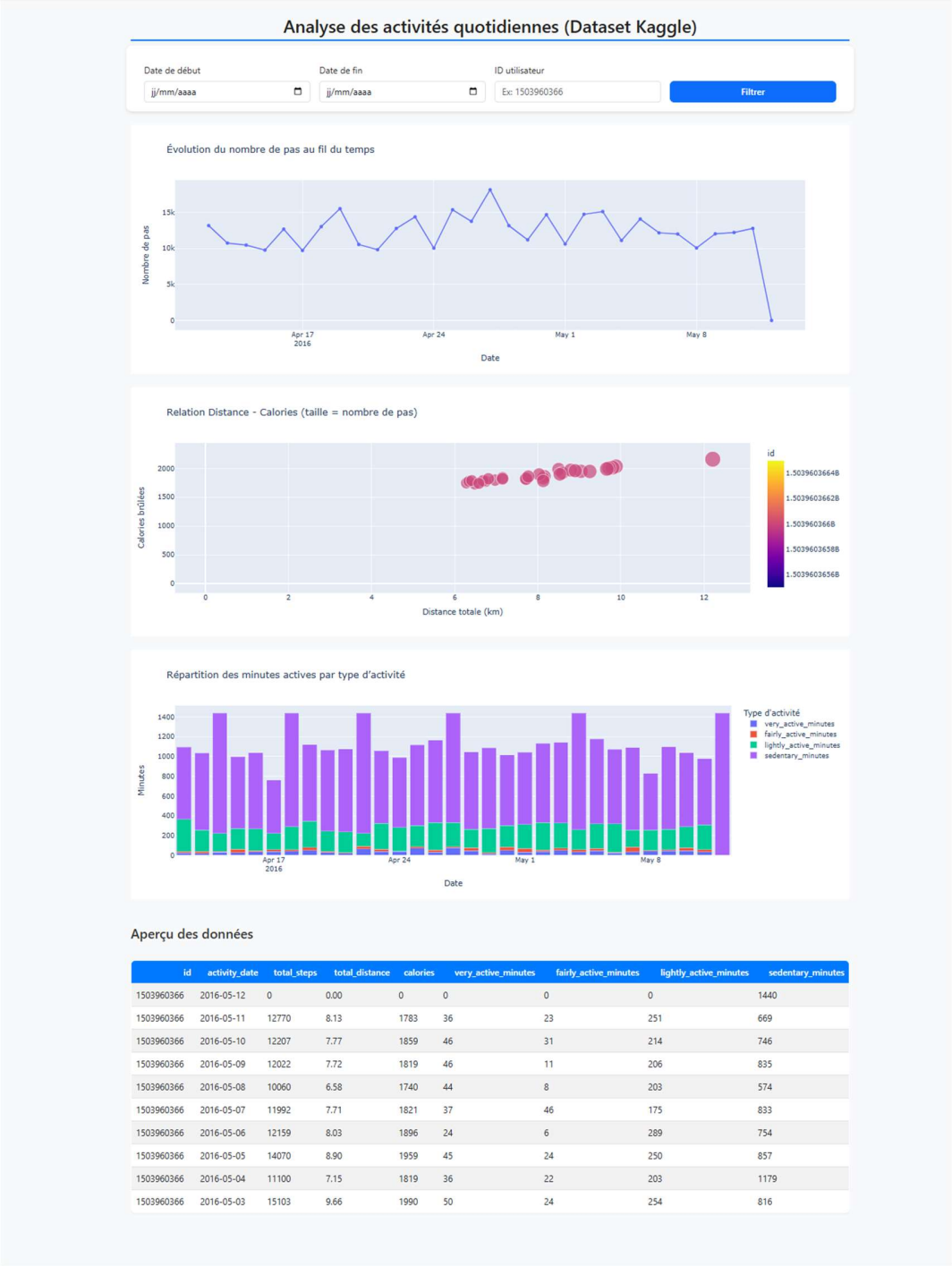
6.1 With Fitbit Auth 2.0

The Fitbit account used in this project was empty, that is why there is no data displayed on the following screenshot.



6.2 Without Fitbit Auth 2.0 (Fitbit Kaggle data)

In the following example, a Fitbit dataset downloaded from Kaggle in **CSV** format was imported into a PostgreSQL database using the *psycopg2* library. This approach eliminated the need to retrieve data from Fitbit through authentication. The Flask application was then connected to the database to visualize the data using interactive charts.



	id	activity_date	total_steps	total_distance	calories	very_active_minutes	fairly_active_minutes	lightly_active_minutes	sedentary_minutes
1503960366	2016-05-12	0	0.00	0	0	0	0	1440
1503960366	2016-05-11	12770	8.13	1783	36	23	251	669
1503960366	2016-05-10	12207	7.77	1859	46	31	214	746
1503960366	2016-05-09	12022	7.72	1819	46	11	206	835
1503960366	2016-05-08	10060	6.58	1740	44	8	203	574
1503960366	2016-05-07	11992	7.71	1821	37	46	175	833
1503960366	2016-05-06	12159	8.03	1896	24	6	289	754
1503960366	2016-05-05	14070	8.90	1959	45	24	250	857
1503960366	2016-05-04	11100	7.15	1819	36	22	203	1179
1503960366	2016-05-03	15103	9.66	1990	50	24	254	816